

Punto 3.7 - Joins y navegación en consultas

Práctica

Ejercicio: Añadir consultas con joins y navegación al repositorio de órdenes del proyecto AceriaData. Crear métodos que combinen órdenes con planchas, órdenes con detalle, órdenes con aleaciones y órdenes con múltiples entidades. Crear un caso de uso que ejecute estas consultas y analice el SQL generado.

Contexto del proyecto: En el punto 3.6 se añadieron agrupaciones con proyección, incluyendo colecciones internas y filtros Having. En este punto se añaden consultas con joins y navegación, incluyendo Join, SelectMany, DefaultIfEmpty y Include. Estas consultas se usarán en el punto 3.8 para la carga Eager y en el punto 3.9 para la carga Lazy.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO OrdenConPlanchasYDetalleDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConPlanchasYDetalleDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenConPlanchasYDetalleDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
    public DetalleDto? Detalle { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres. Línea 3: public class OrdenConPlanchasYDetalleDto → declara el DTO. Línea 5: public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 6: public string Cliente { get; set; } = string.Empty; → cliente. Línea 7: public string Estado { get; set; } = string.Empty; → estado. Línea 8: public List Planchas { get; set; } = new(); → colección de planchas. Línea 9: public DetalleDto? Detalle { get; set; } → detalle opcional.

Error común: si el DTO no marca la propiedad Detalle como anulable, el compilador lanza un warning. Se debe marcar como ? si puede ser null.

Paso 3: Crear el DTO OrdenConAleacionesDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConAleacionesDto.cs:

```

namespace AceriaData.Application.Dtos;

public class OrdenConAleacionesDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public List<AleacionDto> Aleaciones { get; set; } = new();
}

public class AleacionDto
{
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public decimal PorcentajeCarbono { get; set; }
    public decimal PorcentajeManganeso { get; set; }
    public decimal CantidadUtilizada { get; set; }
}

```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres. Línea 3: public class OrdenConAleacionesDto → declara el DTO. Línea 5: public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 6: public string Cliente { get; set; } = string.Empty; → cliente. Línea 7: public List<AleacionDto> Aleaciones { get; set; } = new(); → colección de aleaciones. Línea 10: public class AleacionDto → declara el DTO de aleación. Línea 12: public string Nombre { get; set; } = string.Empty; → nombre. Línea 13: public stringCodigo { get; set; } = string.Empty; → código. Línea 14: public decimal PorcentajeCarbono { get; set; } → porcentaje de carbono. Línea 15: public decimal PorcentajeManganeso { get; set; } → porcentaje de manganeso. Línea 16: public decimal CantidadUtilizada { get; set; } → cantidad utilizada.

Error común: si la colección no se inicializa con new(), se produce una NullReferenceException al intentar añadir elementos.

| Paso 4: Añadir los métodos de join a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```

using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime
hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();
    List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas();
    List<OrdenConDetalleDto> ObtenerOrdenesConDetalle();
    List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados();
    List<OrdenCompletaDto> ObtenerOrdenesCompletas();

    int ContarOrdenes();
    int ContarOrdenesPorEstado(string estado);
    bool ExisteAlgunaOrden();
    bool TodasLasOrdenesPendientes();
    decimal ObtenerPesoTotalDePlanchas();
    double ObtenerPesoPromedioDePlanchas();
    decimal ObtenerPesoMinimoDePlanchas();
    decimal ObtenerPesoMaximoDePlanchas();
    DateTime ObtenerFechaMasAntigua();
    DateTime ObtenerFechaMasReciente();

    List<ResumenPorClienteDto> ObtenerResumenPorCliente();
    List<ResumenPorEstadoDto> ObtenerResumenPorEstado();
    List<ResumenPorClienteDto> ObtenerClientesConMasDeUnaOrden();
    List<ResumenMensualDto> ObtenerResumenMensual();
    List<ResumenPorClienteConOrdenesDto> ObtenerResumenPorClienteConOrdenes();
    List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstado();
    List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstadoConFiltro();
    List<ResumenPorClienteDto> ObtenerResumenPorClienteConMultiplesAgregaciones();

    List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalle();
    List<OrdenConAleacionesDto> ObtenerOrdenesConAleaciones();
    List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalleConJoinExplicito();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs. Línea 2: using AceriaData.Domain.Entities; → importa las entidades. Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres. Línea 6: public interface IOOrdenRepositorio → declara la interfaz. Línea 8: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id. Línea 9: List ObtenerTodas(); → método que obtiene todas. Línea 10: IQueryable ObtenerQueryable(); → método que devuelve IQueryable. Línea 11: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número. Línea 12: List ObtenerPendientesPorCliente(string cliente); → método que obtiene las pendientes de un cliente. Línea 13: List ObtenerPorEstadoOrdenadasPorFecha(string estado); → método que obtiene por estado ordenadas. Línea 14: List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta); → método que obtiene por rango de fechas. Línea 15: List ObtenerPorClienteOrdenadas(string cliente); → método que obtiene por cliente ordenadas. Línea 16: List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta); → método que combina cliente y rango. Línea 18: List ObtenerClientesUnicos();

→ método que obtiene los clientes únicos. Línea 19: `List ObtenerResumenes();` → método que obtiene los resúmenes. Línea 20: `List ObtenerResumenesPorEstado(string estado);` → método que obtiene los resúmenes por estado. Línea 21: `List ObtenerOrdenesConTotales();` → método que obtiene las órdenes con totales. Línea 22: `List ObtenerOrdenesConPlanchas();` → método que obtiene las órdenes con planchas. Línea 23: `List ObtenerOrdenesConDetalle();` → método que obtiene las órdenes con detalle. Línea 24: `List ObtenerOrdenesConTotalesDetallados();` → método que obtiene las órdenes con totales detallados. Línea 25: `List ObtenerOrdenesCompletas();` → método que obtiene las órdenes completas. Línea 27: `int ContarOrdenes();` → método que cuenta las órdenes. Línea 28: `int ContarOrdenesPorEstado(string estado);` → método que cuenta las órdenes por estado. Línea 29: `bool ExisteAlgunaOrden();` → método que comprueba si existe alguna orden. Línea 30: `bool TodasLasOrdenesPendientes();` → método que comprueba si todas las órdenes están pendientes. Línea 31: `decimal ObtenerPesoTotalDePlanchas();` → método que obtiene el peso total. Línea 32: `double ObtenerPesoPromedioDePlanchas();` → método que obtiene el peso promedio. Línea 33: `decimal ObtenerPesoMinimoDePlanchas();` → método que obtiene el peso mínimo. Línea 34: `decimal ObtenerPesoMaximoDePlanchas();` → método que obtiene el peso máximo. Línea 35: `DateTime ObtenerFechaMasAntigua();` → método que obtiene la fecha más antigua. Línea 36: `DateTime ObtenerFechaMasReciente();` → método que obtiene la fecha más reciente. Línea 38: `List ObtenerResumenPorCliente();` → método que obtiene el resumen por cliente. Línea 39: `List ObtenerResumenPorEstado();` → método que obtiene el resumen por estado. Línea 40: `List ObtenerClientesConMasDeUnaOrden();` → método que obtiene los clientes con más de una orden. Línea 41: `List ObtenerResumenMensual();` → método que obtiene el resumen mensual. Línea 42: `List ObtenerResumenPorClienteConOrdenes();` → método que obtiene el resumen por cliente con órdenes. Línea 43: `List ObtenerResumenPorClienteYEstado();` → método que obtiene el resumen por cliente y estado. Línea 44: `List ObtenerResumenPorClienteYEstadoConFiltro();` → método que obtiene el resumen con filtro. Línea 45: `List ObtenerResumenPorClienteConMultiplesAgregaciones();` → método que obtiene el resumen con múltiples agregaciones. Línea 47: `List ObtenerOrdenesConPlanchasYDetalle();` → método que combina órdenes con planchas y detalle. Línea 48: `List ObtenerOrdenesConAleaciones();` → método que combina órdenes con aleaciones. Línea 49: `List ObtenerOrdenesConPlanchasYDetalleConJoinExplicito();` → método que usa Join explícito. Línea 51: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden. Línea 52: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

| Paso 5: Implementar los métodos de join en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalle()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
}
```

```

    })
    .ToList();
}

public List<OrdenConAleacionesDto> ObtenerOrdenesConAleaciones()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConAleacionesDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Aleaciones = o.OrdenesAleaciones.Select(oa => new AleacionDto
            {
                Nombre = oa.Aleacion.Nombre,
               Codigo = oa.Aleacion.Codigo,
                PorcentajeCarbono = oa.Aleacion.PorcentajeCarbono,
                PorcentajeManganeso = oa.Aleacion.PorcentajeManganeso,
                CantidadUtilizada = oa.CantidadUtilizada
            }).ToList()
        })
        .ToList();
}

public List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalleConJoinExplicito()
{
    return _context.OrdenesFabricacion
        .GroupJoin(
            _context.PlanchasAcero,
            o => o.Id,
            p => p.OrdenId,
            (o, planchas) => new { Orden = o, Planchas = planchas })
        .GroupJoin(
            _context.DetallesOrden,
            op => op.Orden.Id,
            d => d.OrdenId,
            (op, detalles) => new { op.Orden, op.Planchas, Detalle = detalles.FirstOrDefault() })
        .OrderBy(x => x.Orden.NumeroOrden)
        .Select(x => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = x.Orden.NumeroOrden,
            Cliente = x.Orden.Cliente,
            Estado = x.Orden.Estado,
            Planchas = x.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = x.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = x.Detalle.ComposicionQuimica,
                TemperaturaColada = x.Detalle.TemperaturaColada,
                Notas = x.Detalle.Notas
            }
        })
        .ToList();
}

```

Línea 1: public List ObtenerOrdenesConPlanchasYDetalle() → declara el método que combina órdenes con planchas y detalle. Línea 3: return _context.OrdenesFabricacion → inicia la consulta. Línea 4: .OrderBy(o => o.NumeroOrden) → ordena por número de orden. Línea 5: .Select(o => new OrdenConPlanchasYDetalleDto → proyecta al DTO. Línea 6: NumeroOrden = o.NumeroOrden, → asigna el número. Línea 7: Cliente = o.Cliente, → asigna el cliente. Línea 8: Estado = o.Estado, → asigna el estado. Línea 9: Planchas = o.Planchas.Select(p => new PlanchaDto → proyecta la colección de planchas. Línea 10: Id = p.Id, → asigna el Id. Línea 11: Espesor = p.Espesor, → asigna el espesor. Línea 12: Ancho = p.Ancho, → asigna el ancho. Línea 13: Largo = p.Largo, → asigna el largo. Línea 14: Peso = p.Peso, →

asigna el peso. Línea 15: `Activa = p.Activa` → asigna el estado activo. Línea 16: `}).ToList()`, → materializa la colección de planchas. Línea 17: `Detalle = o.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe y lo proyecta. Línea 18: `ComposicionQuimica = o.Detalle.ComposicionQuimica`, → asigna la composición química. Línea 19: `TemperaturaColada = o.Detalle.TemperaturaColada`, → asigna la temperatura. Línea 20: `Notas = o.Detalle.Notas` → asigna las notas. Línea 21: `}` → cierra la proyección del detalle. Línea 22: `})` → cierra la proyección. Línea 23: `.ToList()`; → materializa la consulta. Línea 26: `public List ObtenerOrdenesConAleaciones()` → declara el método que combina órdenes con aleaciones. Línea 28: `return _context.OrdenesFabricacion` → inicia la consulta. Línea 29: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden. Línea 30: `.Select(o => new OrdenConAleacionesDto` → proyecta al DTO. Línea 31: `NumeroOrden = o.NumeroOrden`, → asigna el número. Línea 32: `Cliente = o.Cliente`, → asigna el cliente. Línea 33: `Aleaciones = o.OrdenesAleaciones.Select(oa => new AleacionDto` → proyecta la colección de aleaciones. Línea 34: `Nombre = oa.Aleacion.Nombre`, → asigna el nombre. Línea 35: `Codigo = oa.Aleacion.Codigo`, → asigna el código. Línea 36: `PorcentajeCarbono = oa.Aleacion.PorcentajeCarbono`, → asigna el porcentaje de carbono. Línea 37: `PorcentajeManganeso = oa.Aleacion.PorcentajeManganeso`, → asigna el porcentaje de manganeso. Línea 38: `CantidadUtilizada = oa.CantidadUtilizada` → asigna la cantidad utilizada. Línea 39: `}).ToList()` → materializa la colección de aleaciones. Línea 40: `})` → cierra la proyección. Línea 41: `.ToList()`; → materializa la consulta. Línea 44: `public List ObtenerOrdenesConPlanchasYDetalleConJoinExplicito()` → declara el método con Join explícito. Línea 46: `return _context.OrdenesFabricacion` → inicia la consulta. Línea 47: `.GroupJoin(` → primer GroupJoin con planchas. Línea 48: `_context.PlanchasAcero`, → tabla de planchas. Línea 49: `o => o.Id`, → clave del lado izquierdo. Línea 50: `p => p.OrdenId`, → clave del lado derecho. Línea 51: `(o, planchas) => new { Orden = o, Planchas = planchas }` → proyecta la combinación. Línea 52: `.GroupJoin(` → segundo GroupJoin con detalles. Línea 53: `_context.DetallesOrden`, → tabla de detalles. Línea 54: `op => op.Orden.Id`, → clave del lado izquierdo. Línea 55: `d => d.OrdenId`, → clave del lado derecho. Línea 56: `(op, detalles) => new { op.Orden, op.Planchas, Detalle = detalles.FirstOrDefault() }` → proyecta la combinación. Línea 57: `.OrderBy(x => x.Orden.NumeroOrden)` → ordena por número de orden. Línea 58: `.Select(x => new OrdenConPlanchasYDetalleDto` → proyecta al DTO. Línea 59: `NumeroOrden = x.Orden.NumeroOrden`, → asigna el número. Línea 60: `Cliente = x.Orden.Cliente`, → asigna el cliente. Línea 61: `Estado = x.Orden.Estado`, → asigna el estado. Línea 62: `Planchas = x.Planchas.Select(p => new PlanchaDto` → proyecta las planchas. Línea 69: `}).ToList()`, → materializa la colección. Línea 70: `Detalle = x.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe. Línea 77: `})` → cierra la proyección. Línea 78: `.ToList()`; → materializa la consulta.

Error común: si se usa `GroupJoin` sin entender su semántica, el resultado puede ser confuso. Se recomienda usar `SelectMany` con `DefaultIfEmpty` para los LEFT JOIN.

Paso 6: Crear el caso de uso de joins

Crear el archivo `src/AceriaData.Application/UseCases/JoinsUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class JoinsUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public JoinsUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== JOINS Y NAVEGACIÓN ===");

        DemostrarOrdenesConPlanchasYDetalle();
        DemostrarOrdenesConAleaciones();
        DemostrarJoinExplicito();
        DemostrarSqlJoin();
    }
}
```

```

private void DemostrarOrdenesConPlanchasYDetalle()
{
    Console.WriteLine("\n--- Órdenes con planchas y detalle (navegación por propiedades) ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalle();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        Console.WriteLine($"      Planchas: {orden.Planchas.Count}");
        var detalle = orden.Detalle == null ? "Sin detalle" :
        $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";
        Console.WriteLine($"      Detalle: {detalle}");
    }
}

private void DemostrarOrdenesConAleaciones()
{
    Console.WriteLine("\n--- Órdenes con aleaciones (navegación por propiedades) ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConAleaciones();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente}");
        foreach (var aleacion in orden.Aleaciones)
        {
            Console.WriteLine($"      {aleacion.Nombre} ({aleacion.Codigo}) | C:
{aleacion.PorcentajeCarbono}% | Mn: {aleacion.PorcentajeManganeso}% | Cantidad:
{aleacion.CantidadUtilizada} kg");
        }
    }
}

private void DemostrarJoinExplicito()
{
    Console.WriteLine("\n--- Órdenes con planchas y detalle (Join explícito) ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleConJoinExplicito();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        Console.WriteLine($"      Planchas: {orden.Planchas.Count}");
    }
}

private void DemostrarSqlJoin()
{
    Console.WriteLine("\n--- SQL generado por una consulta con navegación ---");

    var consulta = _unidad.Ordenes.ObtenerQueryable()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new
        {
            o.NumeroOrden,
            o.Cliente,
            Planchas = o.Planchas.Select(p => new { p.Espesor, p.Peso }).ToList(),
            Detalle = o.Detalle == null ? null : new { o.Detalle.ComposicionQuimica,
o.Detalle.TemperaturaColada }
        });

    var sql = consulta.ToQueryString();
    Console.WriteLine(sql);
}
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación. Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres. Línea 5: public class JoinsUseCase → declara el caso de uso. Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo. Línea 9: public JoinsUseCase(IUnidadDeTrabajo unidad) → constructor. Línea 11: _unidad = unidad; → asigna el parámetro al campo. Línea 14: public void Ejecutar() → declara el método principal. Línea 16: Console.WriteLine("=== JOINS Y NAVEGACIÓN ==="); → muestra la cabecera. Línea 18: DemostrarOrdenesConPlanchasYDetalle(); → llama al método de órdenes con planchas y detalle.

Línea 19: `DemostrarOrdenesConAleaciones();` → llama al método de órdenes con aleaciones. Línea 20: `DemostrarJoinExplicito();` → llama al método de join explícito. Línea 21: `DemostrarSqlJoin();` → llama al método de SQL de join. Línea 24: `private void DemostrarOrdenesConPlanchasYDetalle()` → declara el método. Línea 26: `Console.WriteLine("\n--- Órdenes con planchas y detalle (navegación por propiedades) ---");` → muestra la cabecera. Línea 28: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalle();` → llama al método del repositorio. Línea 29: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 31: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos de la orden. Línea 32: `Console.WriteLine($" Planchas: {orden.Planchas.Count}");` → muestra el número de planchas. Línea 33: `var detalle = orden.Detalle == null ? "Sin detalle" : $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";` → comprueba si el detalle existe. Línea 34: `Console.WriteLine($" Detalle: {detalle}");` → muestra el detalle. Línea 38: `private void DemostrarOrdenesConAleaciones()` → declara el método. Línea 40: `Console.WriteLine("\n--- Órdenes con aleaciones (navegación por propiedades) ---");` → muestra la cabecera. Línea 42: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConAleaciones();` → llama al método del repositorio. Línea 43: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 45: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente}");` → muestra los datos de la orden. Línea 46: `foreach (var aleacion in orden.Aleaciones)` → itera sobre las aleaciones. Línea 48: `Console.WriteLine($" {aleacion.Nombre} ({aleacion.Codigo}) | C: {aleacion.PorcentajeCarbono}% | Mn: {aleacion.PorcentajeManganeso}% | Cantidad: {aleacion.CantidadUtilizada} kg");` → muestra los datos de la aleación. Línea 52: `private void DemostrarJoinExplicito()` → declara el método. Línea 54: `Console.WriteLine("\n--- Órdenes con planchas y detalle (Join explícito) ---");` → muestra la cabecera. Línea 56: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleConJoinExplicito();` → llama al método del repositorio. Línea 57: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 59: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos. Línea 60: `Console.WriteLine($" Planchas: {orden.Planchas.Count}");` → muestra el número de planchas. Línea 64: `private void DemostrarSqlJoin()` → declara el método. Línea 66: `Console.WriteLine("\n--- SQL generado por una consulta con navegación ---");` → muestra la cabecera. Línea 68: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta. Línea 69: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden. Línea 70: `.Select(o => new` → proyecta a un tipo anónimo. Línea 71: `o.NumeroOrden,` → incluye el número de orden. Línea 72: `o.Cliente,` → incluye el cliente. Línea 73: `Planchas = o.Planchas.Select(p => new { p.Espesor, p.Peso }).ToList(),` → proyecta la colección de planchas. Línea 74: `Detalle = o.Detalle == null ? null : new { o.Detalle.ComposicionQuimica, o.Detalle.TemperaturaColada }` → comprueba si el detalle existe y lo proyecta. Línea 75: `});` → cierra la proyección. Línea 77: `var sql = consulta.ToQueryString();` → obtiene el SQL. Línea 78: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

Paso 7: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<JoinsUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

Paso 8: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<JoinsUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito. Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso. Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 9: Insertar datos de prueba con planchas, detalle y aleaciones

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes, planchas, detalle y aleaciones:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045", PorcentajeCarbono = 0.45m,
    PorcentajeManganeso = 0.75m, Activo = true };
    var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140", PorcentajeCarbono = 0.40m,
    PorcentajeManganeso = 0.85m, Activo = true };
    context.Aleaciones.AddRange(aleacion1, aleacion2);
    context.SaveChanges();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del
    Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur",
    Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    context.OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000,
    Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500,
    Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000,
    Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
    TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    var ordenAleacion1 = new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion1.Id,
    CantidadUtilizada = 1500.5m, EstadoRelacion = "Activa" };
    var ordenAleacion2 = new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion2.Id,
    CantidadUtilizada = 800.0m, EstadoRelacion = "Activa" };
    var ordenAleacion3 = new OrdenAleacion { OrdenFabricacionId = orden2.Id, AleacionId = aleacion1.Id,
    CantidadUtilizada = 1200.0m, EstadoRelacion = "Activa" };
    context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);

    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito. Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`. Línea 4: `context.Database.EnsureDeleted();` → elimina la base. Línea 5: `context.Database.EnsureCreated();` → crea la base. Línea 7: `var aleacion1 = new Aleacion { ... };` → crea la primera aleación. Línea 8: `var aleacion2 = new Aleacion { ... };` → crea la segunda aleación. Línea 9: `context.Aleaciones.AddRange(aleacion1, aleacion2);` → registra las aleaciones. Línea 10: `context.SaveChanges();` → inserta las aleaciones. Línea

12: `var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.` Línea 13: `var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.` Línea 14: `context.OrdenesFabricacion.AddRange(orden1, orden2); → registra las órdenes.` Línea 15: `context.SaveChanges(); → inserta las órdenes.` Línea 17: `var plancha1 = new PlanchaAcero { ... }; → crea la primera plancha.` Línea 18: `var plancha2 = new PlanchaAcero { ... }; → crea la segunda plancha.` Línea 19: `var plancha3 = new PlanchaAcero { ... }; → crea la tercera plancha.` Línea 20: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3); → registra las planchas.` Línea 22: `var detalle1 = new DetalleOrden { ... }; → crea el detalle.` Línea 23: `context.DetallesOrden.Add(detalle1); → registra el detalle.` Línea 25: `var ordenAleacion1 = new OrdenAleacion { ... }; → crea la primera relación.` Línea 26: `var ordenAleacion2 = new OrdenAleacion { ... }; → crea la segunda relación.` Línea 27: `var ordenAleacion3 = new OrdenAleacion { ... }; → crea la tercera relación.` Línea 28: `context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3); → registra las relaciones.` Línea 30: `context.SaveChanges(); → inserta las planchas, el detalle y las relaciones.`

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 10: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada consulta. Se observan las órdenes con planchas y detalle, las órdenes con aleaciones, el join explícito y el SQL generado por una consulta con navegación.

Paso 11: Analizar la salida

La salida del programa muestra información como la siguiente:

=== JOINS Y NAVEGACIÓN ===

--- Órdenes con planchas y detalle (navegación por propiedades) ---

OF-2024-0001 | Constructora del Norte | Pendiente

Planchas: 2

Detalle: C: 0.45%, Mn: 0.75% | 1550.5°C

OF-2024-0002 | Constructora del Sur | Pendiente

Planchas: 1

Detalle: Sin detalle

--- Órdenes con aleaciones (navegación por propiedades) ---

OF-2024-0001 | Constructora del Norte

AISI 1045 (A1045) | C: 0.45% | Mn: 0.75% | Cantidad: 1500.5 kg

AISI 4140 (A4140) | C: 0.40% | Mn: 0.85% | Cantidad: 800.0 kg

OF-2024-0002 | Constructora del Sur

AISI 1045 (A1045) | C: 0.45% | Mn: 0.75% | Cantidad: 1200.0 kg

--- Órdenes con planchas y detalle (Join explícito) ---

OF-2024-0001 | Constructora del Norte | Pendiente

Planchas: 2

OF-2024-0002 | Constructora del Sur | Pendiente

Planchas: 1

--- SQL generado por una consulta con navegación ---

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso], [p].[Id], [d].[ComposicionQuimica],
[d].[TemperaturaColada]
```

```
FROM [OrdenesFabricacion] AS [o]
```

```
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
```

```
LEFT JOIN [DetallesOrden] AS [d] ON [o].[Id] = [d].[OrdenId]
```

```
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
```

```
ORDER BY [o].[NumeroOrden], [o].[Id], [p].[Id]
```

La primera sección muestra las órdenes con planchas y detalle. La segunda sección muestra las órdenes con aleaciones. La tercera sección muestra el join explícito. La cuarta sección muestra el SQL generado por una consulta con navegación.

Observaciones: el SQL generado incluye dos LEFT JOIN con las tablas de planchas y detalles. El filtro global de Soft Delete se aplica automáticamente. El orden de las cláusulas es el correcto: SELECT, FROM, LEFT JOIN, WHERE, ORDER BY.

Paso 12: Diagnosticar un error común

Modificar el método ObtenerOrdenesConPlanchasYDetalle para acceder a una propiedad del detalle sin comprobar si existe:

csharp

```
public List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalle()
{
```

```
    return _context.OrdenesFabricacion
```

```
        .OrderBy(o => o.NumeroOrden)
```

```
        .Select(o => new OrdenConPlanchasYDetalleDto
```

```
        {
```

```
            NumeroOrden = o.NumeroOrden,
```

```
            Cliente = o.Cliente,
```

```
            Estado = o.Estado,
```

```
            Planchas = o.Planchas.Select(p => new PlanchaDto
```

```
            {
```

```
                Id = p.Id,
```

```
                Espesor = p.Espesor,
```

```
                Ancho = p.Ancho,
```

```
                Largo = p.Largo,
```

```
                Peso = p.Peso,
```

```
                Activa = p.Activa
```

```
            }).ToList(),
```

```
            Detalle = new DetalleDto
```

```
            {
```

```
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
```

```
                TemperaturaColada = o.Detalle.TemperaturaColada,
```

```
                Notas = o.Detalle.Notas
```

```
            }
        })
    }).ToList();
}
```

Resultado esperado: si alguna orden no tiene detalle, se produce una `NullReferenceException` en tiempo de ejecución porque `o.Detalle` es null. EF Core no puede traducir la proyección a SQL y lanza una excepción.

Solución: comprobar si `o.Detalle` es null antes de proyectarlo, como en la implementación original.

Errores comunes del ejercicio Error Causa Solución `NullReferenceException` en detalle No se comprobó si el detalle es null Usar operador ternario antes de proyectar `INNER JOIN` en lugar de `LEFT JOIN` Se usó `SelectMany` sin `DefaultIfEmpty` Usar `DefaultIfEmpty` para incluir entidades sin relación Producto cartesiano Se incluyeron varias colecciones Usar `AsSplitQuery` o proyecciones `ThenInclude` sin `Include` Se usó `ThenInclude` sin `Include` previo Usar `Include` antes de `ThenInclude` `GroupJoin` confuso Se usó `GroupJoin` sin entender su semántica Preferir `SelectMany` con `DefaultIfEmpty` N+1 en proyecciones Se proyecta una colección sin `ToList` Añadir `ToList` dentro de la proyección

Retos resueltos: Consulta con navegación de tres niveles

Reto: Crear un método en el repositorio que obtenga las órdenes con sus planchas, su detalle y sus aleaciones en una sola consulta. Añadir el DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO `OrdenCompletaConAleacionesDto`:

```
namespace AceriaData.Application.Dtos;

public class OrdenCompletaConAleacionesDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
    public DetalleDto? Detalle { get; set; }
    public List<AleacionDto> Aleaciones { get; set; } = new();
}
```

Línea 1: `namespace AceriaData.Application.Dtos;` → declara el espacio de nombres. Línea 3: `public class OrdenCompletaConAleacionesDto` → declara el DTO. Línea 5: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden. Línea 6: `public string Cliente { get; set; } = string.Empty;` → cliente. Línea 7: `public string Estado { get; set; } = string.Empty;` → estado. Línea 8: `public List Planchas { get; set; } = new();` → colección de planchas. Línea 9: `public DetalleDto? Detalle { get; set; }` → detalle opcional. Línea 10: `public List Aleaciones { get; set; } = new();` → colección de aleaciones.

Paso 2: Añadir el método a la interfaz `IOrdenRepositorio`:

```
List<OrdenCompletaConAleacionesDto> ObtenerOrdenesCompletasConAleaciones();
```

Línea 1: `List ObtenerOrdenesCompletasConAleaciones();` → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```

public List<OrdenCompletaConAleacionesDto> ObtenerOrdenesCompletasConAleaciones()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenCompletaConAleacionesDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            },
            Aleaciones = o.OrdenesAleaciones.Select(oa => new AleacionDto
            {
                Nombre = oa.Aleacion.Nombre,
               Codigo = oa.Aleacion.Codigo,
                PorcentajeCarbono = oa.Aleacion.PorcentajeCarbono,
                PorcentajeManganeso = oa.Aleacion.PorcentajeManganeso,
                CantidadUtilizada = oa.CantidadUtilizada
            }).ToList()
        }).ToList();
}

```

Línea 1: public List ObtenerOrdenesCompletasConAleaciones() → declara el método. Línea 3: return _context.OrdenesFabricacion → inicia la consulta. Línea 4: .OrderBy(o => o.NumeroOrden) → ordena por número de orden. Línea 5: .Select(o => new OrdenCompletaConAleacionesDto → proyecta al DTO. Línea 6: NumeroOrden = o.NumeroOrden, → asigna el número. Línea 7: Cliente = o.Cliente, → asigna el cliente. Línea 8: Estado = o.Estado, → asigna el estado. Línea 9: Planchas = o.Planchas.Select(p => new PlanchaDto → proyecta las planchas. Línea 17: }).ToList(), → materializa la colección. Línea 18: Detalle = o.Detalle == null ? null : new DetalleDto → comprueba si el detalle existe. Línea 24: }, → cierra la proyección del detalle. Línea 25: Aleaciones = o.OrdenesAleaciones.Select(oa => new AleacionDto → proyecta las aleaciones. Línea 32: }).ToList() → materializa la colección. Línea 33: }) → cierra la proyección. Línea 34: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```

private void DemostrarOrdenesCompletasConAleaciones()
{
    Console.WriteLine("\n--- Órdenes completas con aleaciones ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletasConAleaciones();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        Console.WriteLine($"   Planchas: {orden.Planchas.Count}");
        var detalle = orden.Detalle == null ? "Sin detalle" : $"{orden.Detalle.ComposicionQuimica}";
        Console.WriteLine($"   Detalle: {detalle}");
        Console.WriteLine($"   Aleaciones: {orden.Aleaciones.Count}");
    }
}

```

Línea 1: `private void DemostrarOrdenesCompletasConAleaciones()` → declara el método. Línea 3: `Console.WriteLine("\n--- Órdenes completas con aleaciones ---");` → muestra la cabecera. Línea 5: `var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletasConAleaciones();` → llama al método del repositorio. Línea 6: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 8: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos de la orden. Línea 9: `Console.WriteLine($" Planchas: {orden.Planchas.Count}");` → muestra el número de planchas. Línea 10: `var detalle = orden.Detalle == null ? "Sin detalle" :`
`($"{orden.Detalle.ComposicionQuimica}");` → comprueba si el detalle existe. Línea 11: `Console.WriteLine($" Detalle: {detalle}");` → muestra el detalle. Línea 12: `Console.WriteLine($" Aleaciones: {orden.Aleaciones.Count}");` → muestra el número de aleaciones.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarOrdenesCompletasConAleaciones();
```

Paso 6: Ejecutar dotnet run y verificar que las órdenes se muestran con sus planchas, detalle y aleaciones.

Resultado esperado: las órdenes se muestran con su número, cliente, estado, número de planchas, detalle y número de aleaciones.

Analugía final

Los joins en una acería son como las consultas que el jefe de planta hace al archivo central para combinar información de varias carpetas. En lugar de mirar la carpeta de órdenes, la carpeta de planchas y la carpeta de aleaciones por separado, el jefe pide un informe que combine las tres. El archivo central hace el join: para cada orden, busca sus planchas y sus aleaciones, y las presenta juntas. La navegación por propiedades es como pedir el informe usando las referencias internas de las carpetas: cada orden tiene una lista de planchas y una lista de aleaciones. El Join explícito es como pedir el informe indicando manualmente cómo se relacionan las carpetas. `DefaultIfEmpty` es como pedir que se incluyan las órdenes aunque no tengan planchas. `GroupJoin` es como pedir un informe agrupado por orden. `Include` es como pedir que se adjunten las carpetas relacionadas al informe principal. Así funcionan los joins en EF Core: se combinan datos de varias tablas en una sola consulta, y el resultado se presenta de forma estructurada.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `OrdenConPlanchasYDetalleDto` y `OrdenConAleacionesDto`.

Añadido los métodos de join a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `JoinsUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas, detalle y aleaciones.

Ejecutado las demostraciones de órdenes con planchas y detalle, órdenes con aleaciones, join explícito y SQL generado por navegación.

Diagnosticado el error de no comprobar si el detalle es null.

Creado el DTO `OrdenCompletaConAleacionesDto` y su método.

Conexión con el siguiente punto

En este punto se han añadido consultas con joins y navegación al proyecto `AceriaData`, incluyendo `Join`, `SelectMany`, `DefaultIfEmpty`, `GroupJoin` y navegación por propiedades. Se ha analizado el SQL generado y se ha comprobado que EF Core genera `INNER JOIN` o `LEFT JOIN` según el caso. En el siguiente punto se

estudiará la carga Eager en detalle, incluyendo Include, ThenInclude, AsSplitQuery y sus implicaciones en el rendimiento.
